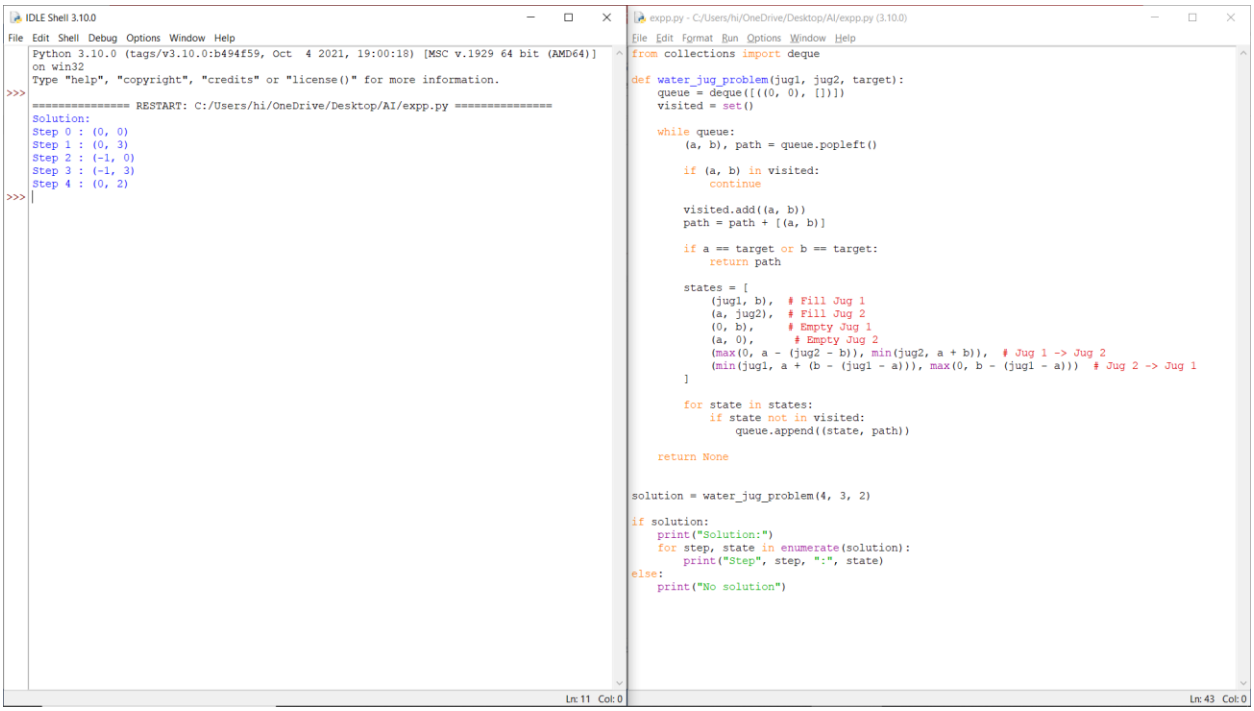
```
from collections import deque

def water_jug_problem(jug1, jug2, target):
    queue = deque([((0, 0), [])])
    visited = set()

    while queue:
        (a, b), path = queue.popleft()

        if (a, b) in visited:
            continue

        visited.add((a, b))
        path = path + [(a, b)]

        if a == target or b == target:
            return path

        states = [
            (jug1, b),  # Fill Jug 1
            (a, jug2),  # Fill Jug 2
            (0, b),     # Empty Jug 1
            (a, 0),     # Empty Jug 2
            (max(0, a - (jug2 - b)), min(jug2, a + b)),  # Jug 1 -> Jug 2
            (min(jug1, a + (b - (jug1 - a))), max(0, b - (jug1 - a)))  # Jug 2 -> Jug 1
        ]

        for state in states:
            if state not in visited:
                queue.append((state, path))

    return None


solution = water_jug_problem(4, 3, 2)

if solution:
    print("Solution:")
    for step, state in enumerate(solution):
        print("Step", step, ":", state)
else:
    print("No solution")
```


IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/hi/OneDrive/Desktop/AI/expp.py ==========
Solution:
Step 0 : (0, 0)
Step 1 : (0, 3)
Step 2 : (-1, 0)
Step 3 : (-1, 3)
Step 4 : (0, 2)
>>>
Ln: 11 Col: 0
expp.py - C:/Users/hi/OneDrive/Desktop/AI/expp.py (3.10.0)
File Edit Format Run Options Window Help
from collections import deque
def water_jug_problem(jug1, jug2, target):
queue = deque([((0, 0), [])])
visited = set()
while queue:
(a, b), path = queue.popleft()
if (a, b) in visited:
continue
visited.add((a, b))
path = path + [(a, b)]
if a == target or b == target:
return path
states = [
(jug1, b), # Fill Jug 1
(a, jug2), # Fill Jug 2
(0, b), # Empty Jug 1
(a, 0), # Empty Jug 2
(max(0, a - (jug2 - b)), min(jug2, a + b)), # Jug 1 -> Jug 2
(min(jug1, a + (b - (jug1 - a))), max(0, b - (jug1 - a))) # Jug 2 -> Jug 1
]
for state in states:
if state not in visited:
queue.append((state, path))
return None
solution = water_jug_problem(4, 3, 2)
if solution:
print("Solution:")
for step, state in enumerate(solution):
print("Step", step, ":", state)
else:
print("No solution")
Ln: 43 Col: 0